

What Determines Long-Context Ability in LLMs?

A Survey with Experimental Analysis

Sihua Zhang, Minghao Xi

May 28, 2026

Abstract

Long-context ability has become a central theme in modern large language models (LLMs), yet the field still lacks a stable answer to a deceptively simple question: what does it actually mean for a model to *use* long context well? Larger context windows alone are not sufficient. A model may accept 128K or even million-token inputs, while still degrading on retrieval from the middle of a document, on multi-hop aggregation across distant spans, or under realistic serving constraints. This report surveys long-context LLMs through a unified lens that separates five factors: nominal context length, effective retrieval length, long-range reasoning ability, inference-time extension effectiveness, and system serving efficiency. We review architectural routes for long-sequence modeling, context extension methods built on top of Transformer backbones, retrieval and memory augmentation, and system techniques such as IO-aware attention and KV-cache management—including token selection and eviction methods such as H₂O, SnapKV, and FIER. We further compare the evaluation landscape, from Needle-in-a-Haystack and position-sensitivity tests to RULER, LongBench, ∞ Bench, BABILong, L-Eval, and LongBench v2. On the empirical side, we already complete a first cross-architecture retrieval study on RULER NIAH tasks using Llama 3.1 8B Instruct and Falcon3-Mamba 7B Instruct, and use its findings to refine the remaining experiment roadmap. The first result is already informative: long-context failure does not emerge as a single smooth curve, but as a task-dependent breakdown in key-value binding, position robustness, and system feasibility. Our main claim is therefore that long-context ability is not a single scalar property but a joint outcome of model design, prompting and retrieval strategy, task type, and system implementation.

1 Introduction

Large language models are increasingly deployed in settings that are inherently long-context: document question answering, multi-document synthesis, repository-scale code understanding, long-horizon agent memory, and structured analysis over extended logs or tables. These scenarios have driven rapid progress in nominal context windows, from a few thousand tokens to 32K, 128K, and even million-token regimes in both commercial and open models. However, the field has also accumulated substantial evidence that accepting longer inputs is not equivalent to using them effectively [3, 4, 12, 14, 18].

This mismatch appears in several ways. Position-sensitive evaluations show that models often underuse information placed in the middle of long prompts [18]. Synthetic benchmarks beyond single-needle retrieval reveal a significant gap between claimed context length and effective context length [12]. Realistic multi-task evaluations suggest that deeper reasoning, rather than nominal window size alone, has become the main bottleneck in long-context settings [4]. At the same time, systems work has shown that long-context deployment is strongly constrained by prefill latency, KV-cache growth, and batching efficiency, even when the underlying model is theoretically capable of longer inputs [9, 15, 32].

This report is organized around a single question: *what determines long-context ability in LLMs?* Instead of treating long context as one monolithic capability, we decompose it into five dimensions:

1. **Nominal context length:** how many tokens the model and serving stack claim to support.
2. **Effective retrieval length:** how far relevant information can be reliably retrieved.
3. **Long-range reasoning ability:** whether the model can aggregate, compare, and reason over distant evidence.
4. **Inference-time extension effectiveness:** how much performance can be improved without retraining the backbone from scratch.
5. **System serving efficiency:** whether the model can be run with acceptable latency, memory, and throughput.

The rest of the report follows this analytical lens. Section 2 formalizes the framework. Sections 3–5 survey modeling, extension, retrieval, and systems techniques. Section 6 reviews benchmarks and what they actually measure. Section 7 reports the first completed experiment family and explains how it reshapes the remaining empirical roadmap. We conclude with a discussion of open challenges and a synthesis of current evidence.

2 Problem Formulation and Analytical Framework

2.1 Why context window size is not enough

Context window size is best understood as a necessary but insufficient condition. A larger window makes it *possible* to provide more evidence to the model, but it says little about whether the model can retrieve the right facts, combine them correctly, or do so under realistic latency and memory budgets. This distinction has become clearer as long-context evaluations evolved from simple passkey or single-needle retrieval to richer tests of aggregation and multi-hop tracing [12, 14, 30].

2.2 A five-dimensional view of long-context ability

We define long-context ability as the model’s capacity to preserve useful performance as input length grows along five coupled dimensions.

Nominal context length. This is the maximum context size advertised by a model or infrastructure stack. It is influenced by positional encoding, training length, implementation choices, and serving software. It answers “can the input be admitted?” but not “can the input be used well?”

Effective retrieval length. This is the largest length regime at which relevant information can still be found reliably. Needle-in-a-Haystack, key-value retrieval, and retrieval-focused RULER tasks are natural measurement tools here [12, 18].

Long-range reasoning ability. This dimension concerns multi-hop chaining, cross-segment aggregation, comparison, and variable tracking over distant evidence. Recent benchmarks increasingly suggest that this is the hardest long-context sub-problem [4, 12, 14].

Inference-time extension effectiveness. Many practical approaches do not redesign the architecture from scratch, but instead modify positional scaling, chunk the input, manage cache selectively, or introduce memory-like mechanisms at inference time [6, 10, 13, 16, 20, 24, 26, 27, 31]. Their value lies in how much effective context they recover per unit implementation complexity.

System serving efficiency. In production and in realistic experiments, long context is constrained by prefill time, decode throughput, peak memory, cache reuse, and batching efficiency. System work is therefore not just a deployment detail; it determines what long-context regimes are empirically reachable [7, 9, 15, 32].

2.3 Main thesis

The core thesis of this report is that long-context ability is a *joint property* of model architecture, inference strategy, task type, and system realization. A model can be strong on one dimension and weak on another. For example, it may preserve single-needle retrieval but fail on aggregation, or it may reason well at moderate length but become unusably slow under realistic serving constraints.

3 Technical Foundations of Long-Context Modeling

3.1 The standard Transformer bottleneck

The modern discussion starts from the success of the Transformer [23]. Full self-attention offers flexible content-based interaction and can be written as

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} \right) V. \quad (1)$$

This expression makes the source of the long-context bottleneck explicit: every token can attend to every other token, so the score matrix $QK^\top$ grows with all pairwise interactions. For a sequence of length n , this leads to quadratic scaling, which is commonly summarized as

$$\text{time} = \mathcal{O}(n^2d), \quad \text{memory} = \mathcal{O}(n^2). \quad (2)$$

These asymptotics explain why simply increasing the context window quickly becomes expensive. In long-context regimes, the bottleneck is not only arithmetic complexity; memory traffic and KV-cache growth become equally important [9].

3.2 Sparse and local-global attention

One classical route is to reduce the interaction graph. Longformer combines local sliding-window attention with task-motivated global tokens, achieving linear scaling in sequence length for many practical settings [5]. BigBird augments local attention with random and global connections and shows that sparse structures can preserve strong theoretical properties while enabling much longer inputs [29]. These models demonstrate that many document tasks do not require fully dense attention at every layer and every position.

A simple way to express local attention is to restrict token i to a neighborhood

$$\mathcal{N}(i) = \{j \mid i - w \leq j \leq i\}, \quad (3)$$

where w is the window size over past context. The key change is that the attention computation no longer ranges over all n positions, but only over a bounded subset for each token. Under this restriction, the complexity becomes

$$\text{time} = \mathcal{O}(nwd), \quad (4)$$

with $w \ll n$ in the long-sequence regime. More general sparse schemes can be viewed as expanding $\mathcal{N}(i)$ with selected global or random connections, but the basic idea is the same: only a limited subset of the full attention graph is retained.

Their main limitation is reduced flexibility in information routing. When the attention pattern is fixed in advance, the model may be unable to directly connect distant tokens that are crucial for a particular instance or task. As a result, sparse and local-global designs primarily improve computational tractability, whereas their effectiveness on complex long-range reasoning depends heavily on how the sparsity pattern is designed and on whether the training setup adequately exposes the model to such dependencies.

3.3 Linear-time, low-rank, and kernelized alternatives

Another route is to approximate or reformulate attention itself. This broad family includes low-rank methods, kernelized attention, and recurrent-style sequence models. While these approaches reduce asymptotic cost, the key practical question is whether they retain the content selectivity that makes Transformers powerful in language modeling.

This trade-off is especially clear in recent work on state-space and recurrent models. A standard discrete-time state-space model can be written as

$$h_t = Ah_{t-1} + Bx_t, \quad y_t = Ch_t, \quad (5)$$

where h_t is the latent state, x_t is the input token representation, and y_t is the output. This form highlights the main appeal of state-space models: information is propagated through a compressed recurrent state rather than through an ever-growing pairwise attention map. Mamba argues that earlier linear-time sequence models remained weaker than Transformers on language tasks partly because their sequence dynamics were not sufficiently selective. Its central idea is to make key components of the state-space dynamics input-dependent:

$$(A_t, B_t, C_t) = g(x_t), \quad h_t = A_th_{t-1} + B_tx_t, \quad y_t = C_th_t, \quad (6)$$

so that the model can modulate what information is retained, updated, or read out at each step [11]. In other words, the recurrence is no longer governed by a fixed transition rule; it is conditioned on the current token. Mamba-2 then reframes the relationship between Transformers and state-space models through structured state space duality, further improving efficiency and implementation simplicity [8]. Hybrid models such as Jamba suggest that the practical frontier may lie not in replacing attention everywhere, but in combining a small amount of precise attention with more efficient recurrent or state-space computation [17]. More recent linear recurrent designs such as Gated DeltaNet further strengthen this line by explicitly targeting retrieval and long-context weaknesses in Mamba-like models [28].

3.4 Implication for long-context research

Architectural work addresses a fundamental question: how can sequence models scale beyond the quadratic regime without giving up too much content-based access? The current evidence suggests a nuanced answer. Pure linear or recurrent models can offer substantial efficiency gains, but the strongest long-context behavior often still benefits from some form of exact or near-exact retrieval path, whether through attention, hybridization, or external memory.

4 Extending Usable Context Without Retraining the Backbone

The previous section focused on architectural changes that modify the sequence model itself. In practice, however, a large share of long-context progress comes from a different strategy: keeping the backbone largely intact while changing how context is represented, accessed, or selected. This family is broader than purely training-free extrapolation. Some methods rely on lightweight fine-tuning or positional recalibration, some reorganize attention during inference, some attach an explicit memory mechanism, and some retrieve only the most relevant evidence before it enters the model context at all. What unifies them is that they aim to expand *usable* context without requiring a completely new backbone.

4.1 Positional information extension

Many practical long-context methods keep the Transformer backbone intact and instead modify the positional mechanism. ALiBi demonstrates that a simple linear distance bias can support test-time extrapolation to longer inputs than seen in training [21]. At attention head h , this can be written as

$$e_{ij}^{(h)} = \frac{q_i^\top k_j}{\sqrt{d}} - m_h(i - j), \quad (7)$$

where m_h is a head-specific slope and the bias grows with relative distance. The formula shows that ALiBi does not replace attention itself; instead, it systematically reshapes the attention scores so that distance is encoded directly in the logits. In the RoPE family, Position Interpolation rescales positions back into the original training range, substantially stabilizing long-context extension with minimal fine-tuning [6]. A simple way to express this idea is

$$p' = p \cdot \frac{L_{\text{train}}}{L_{\text{test}}}, \quad (8)$$

where p is a position in the longer test sequence, L_{train} is the context length seen during training, and L_{test} is the desired extended length. Intuitively, a long test sequence is compressed back into the positional range where the model is better calibrated. More generally, one can write

$$p' = f(p), \quad f : [0, L_{\text{test}}] \rightarrow [0, L_{\text{train}}], \quad (9)$$

which emphasizes that the extended context is mapped back into the positional range. YaRN improves the efficiency of such extension, while LongRoPE pushes this paradigm to much larger windows through non-uniform interpolation and progressive extension [10, 20].

These methods matter because they are much cheaper than training a new long-context backbone from scratch. However, they mainly answer “how do we feed longer inputs into the model?” rather than “how do we ensure the model reasons well once those inputs are available?” The distinction is crucial for interpreting benchmark gains.

4.2 Inference-time long-context adaptation

Attention-structure reorganization. Self-Extend addresses long-context extrapolation by modifying the attention pattern at inference time. Instead of simply increasing the dense attention window or discarding distant tokens, it constructs a bi-level attention pattern: *neighbor attention* preserves fine-grained local interactions within a short range, while *grouped attention* aggregates farther tokens into coarser units [13]. The method therefore tries to preserve local fidelity while maintaining coarse long-range connectivity, without requiring further long-context pretraining.

KV-cache quantization, eviction, and retrieval. A second line of inference-time methods targets the KV cache budget directly. These methods do not primarily ask how to change the model’s attention structure, but how to make the stored keys and values cheaper, smaller, or more selective during generation.

KIVI represents the most direct form of KV-cache compression: it keeps the cache structure intact but reduces the precision of stored keys and values [19]. Its central observation is that the key and value caches have different distributional properties, so they should be quantized asymmetrically: keys are quantized per channel, while values are quantized per token. This makes KIVI a useful baseline for KV-cache efficiency because it compresses the representation of cached information rather than deciding which tokens to evict.

StreamingLLM instead reduces the active cache by retaining only a small set of early *attention sink* tokens together with a recent sliding window [27]. If $\mathcal{S}_{\text{sink}}$ denotes the set of sink-token indices and w is the recent window size, the retained memory at time step t can be expressed as

$$\mathcal{M}_t = \mathcal{S}_{\text{sink}} \cup \{t - w + 1, \dots, t\}. \quad (10)$$

Attention is then computed only over the retained set,

$$\text{Attn}_t = \text{softmax}\left(\frac{q_t K_{\mathcal{M}_t}^\top}{\sqrt{d}}\right) V_{\mathcal{M}_t}. \quad (11)$$

Compared with KIVI, StreamingLLM changes not the numerical precision of the KV cache, but the set of tokens that remain visible to the model.

H₂O further makes cache retention adaptive. It observes that a small subset of tokens, termed *heavy hitters*, accumulate disproportionately large attention scores and are repeatedly referenced during decoding [31]. The cache therefore balances heavy-hitter tokens with a recent window. The eviction decision can be framed as selecting a subset $\mathcal{C}_t \subset \{1, \dots, t\}$ of size B that maximizes accumulated attention mass:

$$\mathcal{C}_t = \arg \max_{|\mathcal{S}|=B} \sum_{i \in \mathcal{S}} \sum_{s \leq t} a_{s,i}, \quad (12)$$

where $a_{s,i}$ is the attention score from decoding step s to token i . This formulation makes the selection criterion explicit: tokens that have been consistently attended to are preferred over tokens that were only briefly relevant.

SnapKV shifts the selection decision to the prefill phase rather than performing it incrementally during decoding [16]. The key observation is that each attention head exhibits a predictable focus pattern: the positions it will attend to during generation can be inferred from a short *observation window* at the end of the prompt. SnapKV uses this window to score and cluster KV positions per head, retaining only high-importance positions before generation begins. This front-loaded compression avoids the overhead of per-step eviction.

FIER pushes the granularity of selection further [24]. Rather than operating at the page or chunk level, it selects individual tokens from the full cache at each decoding step. To make token-level retrieval efficient, FIER uses 1-bit quantized keys as lightweight importance proxies, enabling query-relevant token selection without the cost of full-precision key comparisons.

Taken together, these methods form a spectrum of KV-cache adaptation strategies. KIVI compresses the representation of the cache; StreamingLLM applies a fixed sink-plus-window retention rule; H₂O performs dynamic heavy-hitter eviction; SnapKV front-loads selection into the prefill stage; and FIER performs fine-grained query-aware token retrieval. They differ in *what* is reduced (precision or token set), *when* selection occurs, and *how* importance is estimated.

4.3 Retrieval as context selection for long-context processing

So far, the discussion has focused on inference-time methods that reorganize attention or reduce the active KV cache. A complementary view is that long-context processing is often less about making every token visible at all times, and more about deciding which pieces of context should be surfaced for the current computation. Under this view, retrieval is a form of *context selection*: it constructs a more useful working context before, or during, model reasoning.

InfLLM sits at the boundary between KV-cache adaptation and retrieval-based long-context processing. It stores distant context into additional memory units and retrieves token-relevant units during attention computation [26]. Unlike the eviction methods above, which decide which cached keys and values should remain active within the model’s standard attention mechanism, InfLLM externalizes remote context into a structured memory and selectively brings back relevant units when needed. The distinction matters: KV eviction methods operate entirely within the attention computation and are transparent to the rest of the model, whereas InfLLM introduces an explicit external memory that the model queries. This makes InfLLM closer in spirit to retrieval-augmented approaches, even though it operates at inference time without any offline indexing step.

That boundary—between inference-time memory management and offline retrieval augmentation—is worth making explicit. The methods discussed so far (KIVI, StreamingLLM, H₂O, SnapKV, FIER, InfLLM) all operate at generation time on a single input: they decide how to compress, evict, or selectively access the KV cache for the current sequence. Retrieval-augmented generation, by contrast, typically involves a separate retrieval step that runs before the model sees the query, drawing from an indexed corpus that is independent of the current generation context. This is a fundamentally different intervention point in the long-context pipeline.

This perspective is especially important in multi-document QA, enterprise knowledge access, codebase understanding, and long-form scientific synthesis, where reading everything at once may be either too expensive or too distracting. LongBench already showed that retrieval and compression can help weaker long-context models, although they do not fully substitute for intrinsically stronger long-context understanding [3]. More recent work rethinks the granularity and structure of retrieval itself. RAPTOR organizes documents through recursive abstraction for tree-structured retrieval [22]. LongRAG argues that stronger readers make much larger retrieval units feasible and sometimes preferable [25]. Anthropic’s contextual retrieval further emphasizes that retrieval quality depends on preserving local context around chunks, rather than embedding isolated fragments alone [2].

The key point for this section is that long-context capability is not always achieved by passing more tokens through the model in a single shot. In many settings, it is achieved by deciding which evidence should be surfaced, in what granularity, and in what organization, before the model reasons over it.

4.4 What these strategies actually change

The methods surveyed in this section should not be grouped together merely because they are all cheaper than retraining a new backbone. They intervene at different points in the long-context pipeline and address different failure modes.

Positional extension methods (ALiBi, PI, YaRN, LongRoPE) change *admissible length*: they determine how long an input the model can accept without positional out-of-distribution failure, but say nothing about whether the model uses that input well.

Attention-structure reorganization (Self-Extend) changes *how the model attends* to distant tokens during inference, trading off local precision against coarse long-range coverage.

KV-cache adaptation methods (KIVI, StreamingLLM, H₂O, SnapKV, FIER) change *what is*

retained in memory during generation. They differ in whether they compress representation precision (KIVI), apply a fixed retention rule (StreamingLLM), perform dynamic importance-based eviction (H₂O), front-load selection into prefill (SnapKV), or perform fine-grained per-step token retrieval (FIER).

Inference-time memory augmentation (InfLLM) changes *how remote context is stored and accessed*, externalizing it beyond the standard KV cache while remaining tied to the current generation.

Retrieval augmentation (RAPTOR, LongRAG, contextual retrieval) changes *which evidence enters the context at all*, operating on an offline-indexed corpus before generation begins.

Because these strategies target different failure modes, they should not be evaluated with a single metric, and gains on one dimension do not imply gains on others.

5 Systems for Long-Context LLMs

Long-context research often foregrounds architecture and benchmark accuracy, but real deployment is frequently dominated by system constraints. If prefill is too slow, if KV-cache memory prevents useful batching, or if cache reuse is poor across repeated prefixes, then nominal context support has limited practical value.

5.1 IO-aware attention and kernel optimization

FlashAttention reframed attention as an IO problem, showing that reducing high-bandwidth-memory traffic yields large practical gains without approximating the attention computation [9]. FlashAttention-2 further improves parallelism and work partitioning, pushing exact attention closer to hardware limits [7]. These results matter for long context because they lower the constant factors that otherwise make even exact attention impractical at scale.

5.2 KV-cache management and serving engines

PagedAttention and vLLM address a central bottleneck in autoregressive serving: the dynamic and memory-hungry KV cache. For a Transformer with N_{layers} layers, H heads, head dimension d_{head} , sequence length L , and per-element storage cost b bytes, the KV-cache size can be approximated as

$$M_{\text{KV}} \approx 2 \cdot L \cdot N_{\text{layers}} \cdot H \cdot d_{\text{head}} \cdot b, \quad (13)$$

where the factor of 2 accounts for keys and values. This estimate clarifies why long-context serving rapidly becomes memory-bound even when model weights are fixed: the cache grows linearly with sequence length and with the depth of the model. A complementary high-level view is that, for standard Transformers, prefill scales roughly as

$$\text{Prefill cost} \sim \mathcal{O}(L^2), \quad \text{Decode step cost} \sim \mathcal{O}(L), \quad (14)$$

which explains why long prompts and long generation histories stress the system in different ways: prefill is dominated by prompt length, while decoding remains sensitive to the accumulated context. By managing KV memory like virtual memory pages, vLLM substantially improves throughput, especially at longer sequence lengths [15]. SGLang extends the systems perspective beyond one-shot decoding, introducing RadixAttention for efficient prefix reuse across structured language-model programs [32]. This is especially relevant for agentic and multi-step pipelines, where repeated long prefixes are common.

5.3 Serving Efficiency as a Dimension of Long-Context Ability

The system perspective is not only about how to deploy a long-context model more efficiently; it also changes what should count as long-context ability in the first place. In the analytical framework of this report, serving efficiency is one of the core dimensions of long-context performance rather than a secondary implementation detail. The relevant metrics include time-to-first-token, prefill latency, decode throughput, peak memory, batch scalability, and cache reuse efficiency. These quantities determine which context lengths remain usable in practice, under what cost, and for what type of workload. A model that performs well on long-context benchmarks but becomes prohibitively slow or memory-intensive at deployment time does not fully realize its nominal long-context capability. For this reason, system efficiency should be evaluated alongside retrieval and reasoning rather than after them.

6 Evaluation of Long-Context Ability

6.1 From retrieval sanity checks to deeper reasoning tests

Evaluation has matured from simple retrieval tests to increasingly realistic multi-task reasoning benchmarks. Needle-in-a-Haystack and related key-value retrieval tasks are useful sanity checks, but they are too narrow to define long-context ability by themselves [18]. Indeed, position-sensitivity analysis shows that even models with long context windows may systematically underuse information placed in the middle of prompts [18].

6.2 Synthetic but harder benchmarks

RULER is a major step beyond single-needle testing. It introduces configurable synthetic tasks spanning retrieval, aggregation, and tracing, and explicitly emphasizes the distinction between claimed and effective context size [12]. ∞ Bench pushes evaluation beyond 100K tokens and combines synthetic and realistic tasks, highlighting that current models still struggle in truly long regimes [30]. BABILong makes the reasoning challenge sharper still by scattering relevant facts through extremely long natural text and measuring whether models can chain them reliably [14].

6.3 Realistic long-context benchmarks

LongBench provides a bilingual, multi-task benchmark spanning question answering, summarization, few-shot learning, synthetic tasks, and code completion [3]. L-Eval contributes not only datasets but also a methodological warning: traditional n-gram overlap is often misaligned with long-form answer quality, so long-context evaluation may require stronger judges and better instructions [1]. LongBench v2 raises the bar further by focusing on realistic multi-task problems that demand deeper understanding and reasoning, suggesting that the next bottleneck is not merely feeding longer documents, but reasoning effectively once they are available [4].

6.4 Benchmark-to-capability mapping

An important methodological lesson is that benchmarks measure different slices of long-context ability:

- **Needle-in-a-Haystack / key-value retrieval:** basic retrieval and position robustness.
- **Lost in the Middle:** positional sensitivity under long inputs.

- **RULER retrieval tasks:** effective retrieval length.
- **RULER aggregation and tracing:** long-range reasoning over synthetic evidence.
- **LongBench / LongBench v2:** realistic long-context understanding and reasoning.
- **Serving benchmarks:** practical utility under latency and memory constraints.

This mapping is critical for experiment design. Without it, one may overgeneralize from a benchmark that measures only one sub-skill. It is therefore useful to make the notion of effective context explicit. If $\text{Acc}(L)$ denotes performance at context length L , then one natural definition is

$$L_{\text{eff}}(\tau) = \max\{L : \text{Acc}(L) \geq \tau\}, \quad (15)$$

where τ is a task-dependent target threshold such as 0.8. This definition captures the longest context regime at which performance remains above a chosen quality bar, and therefore formalizes the difference between claimed window size and usable window size. To quantify how sharply performance degrades with length, one may also track a decay slope such as

$$\Delta(L_1, L_2) = \frac{\text{Acc}(L_2) - \text{Acc}(L_1)}{\log L_2 - \log L_1}. \quad (16)$$

This second quantity is useful because two models may have similar accuracy at one length while differing substantially in how rapidly they deteriorate as the context grows.

7 Experimental Analysis

7.1 A first completed experiment: retrieval under controlled interference

Our first completed experiment targets the retrieval dimension of long-context ability. The goal is not to build another broad leaderboard, but to ask a narrower diagnostic question: does degradation first appear on simple single-target lookup, or only when retrieval also requires precise key-value binding under interference?

We compare two representative architectures, Llama-3.1-8B-Instruct and Falcon3-Mamba-7B-Instruct, on two RULER NIAH subtasks. The first, `niah_single_1`, embeds one target key-value pair in highly repetitive noise text. The second, `niah_multikey_1`, embeds four key-value pairs in natural essay text and asks for the value associated with one queried key. This makes the second task substantially harder: it is still retrieval, but now retrieval must compete with distractor keys, distractor numbers, and realistic textual interference.

For Llama, we evaluate 4K, 8K, 16K, and 32K contexts. For Falcon3-Mamba, stable runs are available at 4K, 8K, and 16K. We do not report a formal 32K Mamba result: in the current 24GB RTX 4090 experimental environment, 32K single-GPU inference runs out of memory, while multi-GPU `device_map=auto` is unstable on the Mamba CUDA-kernel path.

Table 1 summarizes the main result.

The contrast is sharp. Llama remains essentially saturated on both tasks across all tested lengths. Falcon3-Mamba, however, is perfect on the single-key task while collapsing on the multi-key task. This shows that “retrieval” is not a single homogeneous capability. A model may preserve one salient target through a long prompt, yet still fail when it must bind the correct key to the correct value among several candidates.

Position-sensitive analysis makes the difference clearer. Figure 1 shows that Llama remains nearly flat across position buckets, whereas Falcon3-Mamba exhibits a strong recency bias on

Table 1: Exp A accuracy (%) on RULER NIAH retrieval tasks. A dash indicates that the configuration is excluded from formal reporting because it is not stable in the current environment.

Model / Task	4K	8K	16K	32K
Llama single-key	99.8	100.0	100.0	100.0
Llama multi-key	100.0	99.8	100.0	99.4
Falcon3-Mamba single-key	100.0	100.0	100.0	—
Falcon3-Mamba multi-key	43.6	28.4	21.4	—

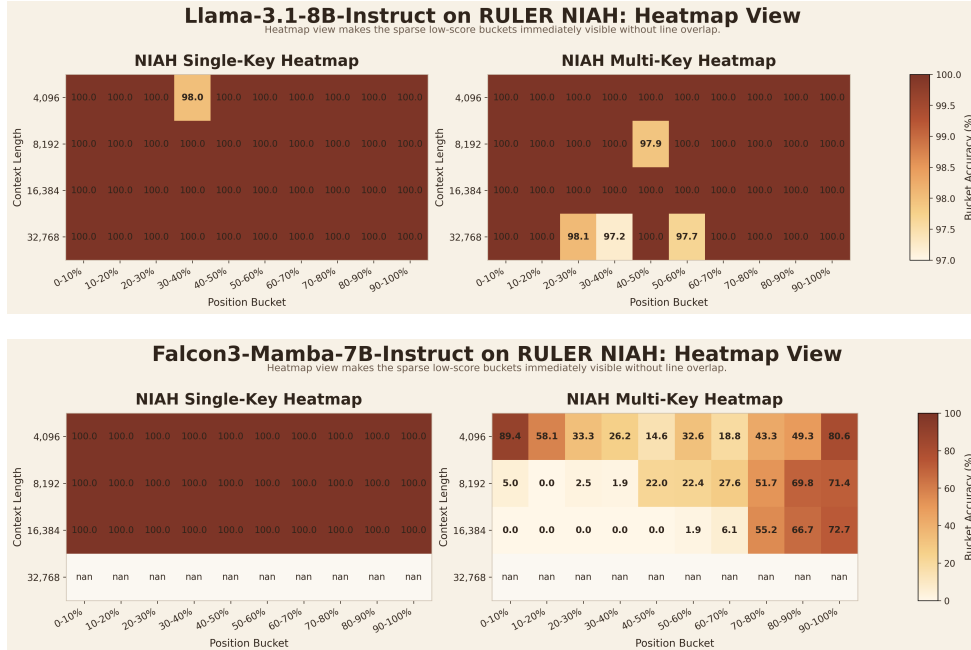


Figure 1: Position-sensitive accuracy heatmaps for Exp A. Top: Llama-3.1-8B-Instruct remains near-saturated across lengths and positions. Bottom: Falcon3-Mamba-7B-Instruct is stable on single-key retrieval but shows severe position sensitivity on multi-key retrieval, especially at longer lengths.

`niah_multikey_1`. At 16K, the early and middle buckets are close to zero, and accuracy recovers only near the end of the sequence. This is more informative than a generic length effect: the model is not merely getting worse with longer inputs, but becoming increasingly dependent on recent evidence.

A second observation is that Falcon3-Mamba is also much slower on the multi-key task. The reason is mostly decoding rather than prefill. On `niah_single_1`, the model usually emits a short answer. On `niah_multikey_1`, failed cases often contain long explanatory generations with an incorrect number, which substantially increases output length and wall-clock time.

The qualitative error pattern is also consistent: in failed multi-key cases, Falcon3-Mamba often outputs a plausible number, but not the *correct* number. In other words, the failure is less about total loss of the prompt and more about unstable key-value binding under interference. This aligns with a broader claim in the literature that state-space and recurrent-style models can be efficient on long sequences, yet remain weaker than attention-based models on exact content addressing and interference-heavy in-context association.

7.2 Implications for the rest of the project

Exp A already changes how the rest of the empirical study should be framed. First, it confirms that benchmark choice matters: if we had evaluated only the single-key setting, we would have concluded that Falcon3-Mamba is essentially perfect on long-context retrieval. Adding the multi-key task reverses that picture. Second, it suggests that the next empirical priority should be to separate retrieval failure from reasoning failure more explicitly. This motivates Exp B: harder RULER reasoning tasks and a small realistic LongBench subset. Third, the failed 32K Mamba run shows that systems feasibility is not an afterthought; it directly shapes what counts as usable long context in practice. This motivates keeping the serving and efficiency dimension in the final paper rather than relegating it to implementation notes.

The current evidence is still narrow: only the retrieval baseline is complete, the model set is small, and Llama is nearly saturated on the present tasks. Even so, Exp A already supports the central thesis of the report: long-context ability is not a single scalar property, but a joint outcome of architecture, task difficulty, interference structure, and hardware-constrained implementation.

8 Discussion and Open Challenges

The survey and the first completed experiment point in the same direction: long-context ability should not be treated as a single number. Even within retrieval, the gap between `niah_single_1` and `niah_multikey_1` is large enough to reverse the qualitative picture for Falcon3-Mamba. This implies that many public discussions around “supports 32K” or “works on long context” remain underspecified unless they also state what kind of retrieval or reasoning is being tested.

A second challenge is that architecture, benchmark, and systems constraints interact in ways that make simple conclusions dangerous. State-space and recurrent models may offer genuine advantages in memory growth and long-sequence efficiency, yet those gains do not automatically translate into strong key-value binding or multi-evidence reasoning. Conversely, Transformers may remain stronger on difficult retrieval and reasoning tasks while becoming bottlenecked by KV-cache cost and prefill latency. The more realistic conclusion is therefore comparative rather than absolute: different architectures fail differently, and these failure modes only become visible when the benchmark suite spans multiple dimensions.

A third challenge is methodological. The current project already encountered a case where nominal model capability and stable experimental capability diverged: Falcon3-Mamba at 32K on the available 24GB 4090 hardware. This experience reinforces a broader point from the systems literature: serving feasibility is not an afterthought. It directly constrains what counts as usable long context for most research groups and practitioners.

The remaining open problem is therefore not simply “how do we make windows even larger?” but rather “how do we preserve exact retrieval, robust reasoning, and acceptable efficiency simultaneously?” Hybrid architectures, stronger inference-time cache selection, retrieval-aware memory systems, and more realistic evaluation protocols are all plausible parts of the answer, but none currently resolves the trade-off on its own.

9 Conclusion

This report surveyed the long-context literature through five dimensions: nominal context length, effective retrieval length, long-range reasoning ability, inference-time extension effectiveness, and system serving efficiency. The literature review suggests that no single technical route—larger

windows, linear-time architectures, cache compression, or retrieval augmentation—fully determines long-context ability on its own. Instead, useful long-context behavior emerges from the interaction between architecture, task, inference strategy, and systems realization.

The first completed experiment in this project already supports that claim. On RULER NIAH retrieval, Llama-3.1-8B-Instruct remains nearly saturated across all tested lengths and both retrieval subtasks. Falcon3-Mamba-7B-Instruct, by contrast, is perfect on the simple single-key task but degrades sharply on the more interference-heavy multi-key task, with strong recency bias and unstable key-value binding. This split shows that even “retrieval” must be decomposed more carefully: success on a weak single-target task does not imply robust long-context retrieval in a harder setting.

The practical lesson is equally important. For Falcon3-Mamba, 32K evaluation is not stably reachable on the current 24GB 4090 setup, which means systems feasibility already shapes the empirical meaning of long-context support. This reinforces the central thesis of the report: long-context ability is not just a property of the model card, but a joint property of benchmark design, model architecture, inference strategy, and hardware-constrained implementation. The remaining experiments in this project—reasoning, inference-time extension, and serving efficiency—are therefore best understood not as independent add-ons, but as the next steps required to complete this multi-dimensional picture.

References

- [1] Chenxin An, Shansan Gong, Ming Zhong, Xingjian Zhao, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. L-Eval: Instituting standardized evaluation for long context language models. *arXiv preprint arXiv:2307.11088*, 2023. URL <https://arxiv.org/abs/2307.11088>.
- [2] Anthropic. Contextual retrieval. Technical blog post, 2024. URL <https://www.anthropic.com/engineering/contextual-retrieval>.
- [3] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2024. URL <https://arxiv.org/abs/2308.14508>.
- [4] Yushi Bai, Xiao Liu, Yuxiang He, et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3756–3780, 2025. URL <https://aclanthology.org/2025.acl-long.183/>.
- [5] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020. URL <https://arxiv.org/abs/2004.05150>.
- [6] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023. URL <https://arxiv.org/abs/2306.15595>.
- [7] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023. URL <https://arxiv.org/abs/2307.08691>.
- [8] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024. URL <https://arxiv.org/abs/2405.21060>.

- [9] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [10] Yiran Ding, Li Lina Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. Longrope: Extending LLM context window beyond 2 million tokens. *arXiv preprint arXiv:2402.13753*, 2024. URL <https://arxiv.org/abs/2402.13753>.
- [11] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. URL <https://arxiv.org/abs/2312.00752>.
- [12] Cheng-Ping Hsieh et al. RULER: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024. URL <https://arxiv.org/abs/2404.06654>.
- [13] Hongyu Jin, Jian Li, Mong Yang, Kun He, et al. LLM maybe longLM: Self-extend LLM context window without tuning. *arXiv preprint arXiv:2401.01325*, 2024. URL <https://arxiv.org/abs/2401.01325>.
- [14] Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. *arXiv preprint arXiv:2406.10149*, 2024. URL <https://arxiv.org/abs/2406.10149>.
- [15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [16] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024. URL <https://arxiv.org/abs/2404.14469>.
- [17] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, et al. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*, 2024. URL <https://arxiv.org/abs/2403.19887>.
- [18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. URL <https://arxiv.org/abs/2307.03172>.
- [19] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32332–32344. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/liu24bz.html>.
- [20] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Edward Shippole. Yarn: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023. URL <https://arxiv.org/abs/2309.00071>.

- [21] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=R8sQPpGCv0>.
- [22] Parth Sarthi, Salman Abdullah, Aman Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. *arXiv preprint arXiv:2401.18059*, 2024. URL <https://arxiv.org/abs/2401.18059>.
- [23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017. URL <https://arxiv.org/abs/1706.03762>.
- [24] Dongwei Wang, Zijie Liu, Song Wang, Yuxin Ren, Jianing Deng, Jingtong Hu, Tianlong Chen, and Huanrui Yang. FIER: Fine-grained and efficient KV cache retrieval for long-context LLM inference. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. URL <https://arxiv.org/abs/2508.08256>.
- [25] Yichuan Wang et al. Longrag: Enhancing retrieval-augmented generation with long-context LLMs. *arXiv preprint arXiv:2406.15319*, 2024. URL <https://arxiv.org/abs/2406.15319>.
- [26] Chao Xiao, Dan Chen, Hanxiao Liu, et al. Infilmm: Training-free long-context extrapolation for LLMs with an efficient context memory. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2402.04617>.
- [27] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023. URL <https://arxiv.org/abs/2309.17453>.
- [28] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024. URL <https://arxiv.org/abs/2412.06464>.
- [29] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33: 17283–17297, 2020. URL <https://arxiv.org/abs/2007.14062>.
- [30] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. Infinitebench: Extending long context evaluation beyond 100k tokens. *arXiv preprint arXiv:2402.13718*, 2024. URL <https://arxiv.org/abs/2402.13718>.
- [31] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H₂O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2306.14048>.
- [32] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024. URL <https://arxiv.org/abs/2312.07104>.